FitTrack Pro — Complete Project Walkthrough

A single-page fitness tracking web application | Built with HTML, CSS, JavaScript (ES6 Modules) | Data stored in localStorage

---

1. Project Overview

FitTrack Pro is a **single-page fitness tracking application** built entirely with vanilla web technologies. It uses **localStorage** as the database — no backend server required. Each user account gets its own isolated data space.

2. File Structure

```
2204/
├── index.html              ← Single HTML page (all sections)
├── styles.css              ← Premium design system + dark mode
├── app.js                  ← Main controller (entry point)
├── README.md               ← Project documentation
└── js/
    ├── storage.js          ← localStorage wrapper (per-user isolation)
    ├── auth.js             ← Login, signup, logout, session
    ├── activity.js         ← CRUD for fitness activities + stats
    ├── goals.js            ← Goal management + progress calculation
    └── ui.js               ← DOM rendering, charts, slider, validation
```

---

3. Detailed File Explanations

3.1 — `index.html` (Single Page)

This is the **only HTML file**. It contains every "page" as a `<section>` element. Only one section is visible at a time, controlled by the `.active` CSS class.

Section ID	Purpose
`authSection`	Login / Signup screen (full-page glassmorphism card)
`home`	Dashboard — stat cards, motivation slider, weekly chart, quick log form
`activities`	Activity log list with filter dropdown, edit & delete
`goals`	Set and track fitness goals with progress bars
`progress`	Monthly calorie chart, performance summary
`community`	Share workout progress posts
`profile`	Profile picture, dark mode toggle, export JSON
`contact`	Support / contact form with email validation

External Libraries (CDN): Bootstrap 5.3 (CSS + JS), Bootstrap Icons, Chart.js, Google Fonts (Inter + Poppins)

3.2 — `styles.css` (Design System)

Uses **CSS Custom Properties** (variables) for theming. All colors, shadows, radii, and transitions are defined as tokens in `:root`. Dark mode overrides these tokens inside `body.dark-mode`.

Key design features:

- Auth screen with animated gradient background and floating orbs
- Cards with 16px radius, shadow lift on hover
- Gradient buttons with glow shadows
- Full-width image slider with overlay captions
- 160+ lines of comprehensive dark mode overrides
- Responsive — sidebar collapses to icons on mobile

3.3 — `js/storage.js` (localStorage Wrapper)

This is the **data layer**. Every read/write to localStorage goes through this module. Key format: `fit_track_{username}_{dataType}`

Method	Purpose
`getUserData(username, key, default)`	Read user-specific data
`setUserData(username, key, value)`	Write user-specific data
`getGlobalData(key, default)`	Read shared data (e.g., user list)
`setGlobalData(key, value)`	Write shared data
`clearUserData(username)`	Delete all data for one user
`getCurrentUser()`	Read who is logged in
`setCurrentUser(username)`	Set/clear current session

3.4 — `js/auth.js` (Authentication)

Manages user accounts stored in localStorage.

Method	What it does
`login(user, pass)`	Checks credentials against stored users, sets session
`signup(user, pass)`	Creates new user in the global users object
`logout()`	Clears current session
`isLoggedIn()`	Returns true if someone is logged in
`getCurrentUsername()`	Returns the logged-in username

> ⚠ **Note:** Passwords are stored in plain text in localStorage. This is for educational purposes only — never do this in production.

3.5 — `js/activity.js` (Activity Tracking)

Manages the fitness activity log with CRUD operations.

Method	What it does
`getAll()`	Returns all activities for current user
`add(data)`	Creates new activity with auto-generated ID

`update(id, data)`	Updates an existing activity
`delete(id)`	Removes an activity by ID
`getStats()`	Calculates totalSteps, totalCalories, avgCalories, bestDay
`getMonthlyCalorieData()`	Groups calories by month (for Chart.js)
`getWeeklyStepData()`	Gets last 7 activities (for trend chart)

Activity object structure:

```
{
    "id": "1707900000000",
    "date": "2026-02-14",
    "steps": 8500,
    "calories": 420,
    "exercise": "Running",
    "timestamp": 1707900000000
}
```

3.6 — `js/goals.js` (Goal Management)

Method	What it does
`getAll()`	Returns all goals for current user
`add(data)`	Creates new goal with auto-generated ID
`delete(id)`	Removes a goal
`calculateProgress(goal)`	Sums all activity data vs. goal target → returns percentage

3.7 — `js/ui.js` (UI Rendering Engine)

The largest module — handles all visual output and interactions.

Method	What it does
`showToast(msg, type)`	Shows a popup notification (auto-hides in 3s)
`showConfirm(title, msg, callback)`	Opens Bootstrap modal for confirmation
`updateCharts()`	Creates/refreshes all Chart.js instances
`destroyChart(id)`	Safely destroys a chart to prevent memory leaks
`initSlider()`	Sets up image slider with auto-play + manual controls
`initInteractions()`	Adds tooltip hover effects to stat cards
`validateForm(formId)`	Validates required fields, email format, positive numbers
`updateNav(target)`	Toggles active class on sidebar links and page sections
`renderActivities(filter)`	Renders activity cards with edit/delete buttons
`renderGoals()`	Renders goal cards with progress bars
`updateSummary()`	Updates dashboard stat numbers

3.8 — `app.js` (Main Controller)

The **entry point**. Imports all modules and wires everything together. Registers all event listeners for auth, navigation, forms, data management, profile, and community.

---

4. How localStorage Works (Deep Dive)

4.1 — What is localStorage?

localStorage is a **browser-based key-value store**. Data persists even after closing the browser. Each key maps to a string value.

```
// Write
localStorage.setItem('myKey', 'myValue');

// Read
const val = localStorage.getItem('myKey'); // → 'myValue'

// Delete
localStorage.removeItem('myKey');
```

Since localStorage only stores **strings**, this app converts objects to JSON:

```
// Saving an array
const data = [{ id: "1", steps: 5000 }];
localStorage.setItem('key', JSON.stringify(data));

// Reading it back
const parsed = JSON.parse(localStorage.getItem('key'));
```

4.2 — Complete localStorage Key Map

Key Pattern	Example	Data Type	Purpose
`fit_track_global_users`	—	Object	All usernames & passwords
`fit_track_current_user`	—	String	Currently logged-in username
`fit_track_{user}_activities`	`fit_track_alice_activities`	Array	Activity logs
`fit_track_{user}_goals`	`fit_track_alice_goals`	Array	Fitness goals
`fit_track_{user}_posts`	`fit_track_alice_posts`	Array	Community posts
`fit_track_profile_img_{user}`	`fit_track_profile_img_alice`	String	Base64 profile image
`fit_track_dark_mode`	—	String	"true" / "false"

4.3 — Data Isolation Per Account

```
localStorage
├── fit_track_global_users        → {"alice":"pass1","bob":"pass2"}
├── fit_track_current_user        → "alice"
```

```
│
├── fit_track_alice_activities     → [{...}, {...}]
├── fit_track_alice_goals          → [{...}]
├── fit_track_alice_posts          → [{...}]
├── fit_track_profile_img_alice    → "..."
│
├── fit_track_bob_activities       → [{...}, {...}]
├── fit_track_bob_goals            → [{...}]
├── fit_track_bob_posts            → [{...}]
├── fit_track_profile_img_bob      → "..."
│
└── fit_track_dark_mode            → "true"
```

> Each user's data lives in separate keys. When Alice logs in, the app reads `fit_track_alice_*`. When Bob logs in, it reads `fit_track_bob_*`. They never interfere with each other.

---

5. Complete Application Roadmap (Login → End)

Step 1 — Page Load
Browser loads `index.html` → `DOMContentLoaded` fires → `app.js` calls `initApp()`.
Checks if a user is already logged in via `Auth.isLoggedIn()`.
If YES → shows main content. If NO → shows the auth screen.

Step 2 — Signup
User enters username + password → clicks "Create Account".
`Auth.signup()` checks if username exists in `fit_track_global_users`.
If new → adds to users object → saves to localStorage → shows success toast.

Step 3 — Login
User enters credentials → clicks "Login".
`Auth.login()` reads `fit_track_global_users`, checks match.
If match → `Storage.setCurrentUser(username)` → saves to `fit_track_current_user`.
`showMainContent()` runs: hides auth, shows sidebar + content, loads per-user profile picture, refreshes all data (stats, charts, activities, goals, posts).

Step 4 — Dashboard (Home)
Stat Cards: Show totalSteps, totalCalories, avgCalories from `Activity.getStats()`.
Motivation Slider: Auto-plays every 5 seconds with manual prev/next/dot controls.
Weekly Steps Chart: Line chart of last 7 activities.
Quick Actions Form: Log new activity (validated before saving).

Step 5 — Log an Activity
Fill in Date, Steps, Calories, Exercise Type.
`UI.validateForm()` checks all required fields and positive numbers.
`Activity.add()` creates object with unique ID (`Date.now()`).
Saved to `fit_track_{username}_activities` → dashboard stats and charts refresh.

Step 6 — Activity Logs Page
All activities shown as cards (newest first).
Filter: Dropdown to show only Running, Walking, Cycling, etc.
Edit: Loads data into dashboard form, deletes old entry.
Delete: Confirmation modal → removes from localStorage.

Step 7 — Goals Page
Set a goal (description, target number, type: steps or calories).
`Goals.add()` saves to `fit_track_{username}_goals`.
Progress bar auto-calculates: sums all activities vs. target.
Delete goals with the X button.

Step 8 — Progress Page
Monthly Calorie Chart: Bar chart grouping calories by month.
Performance Summary: Shows total activities and best day.
Daily Tips: Static motivational text.

Step 9 — Community Page
Write a post → click "Share Post" → saved to `fit_track_{username}_posts`.
Posts show your profile picture, username, and timestamp.
Delete individual posts with the trash button.
Fully isolated: Each account sees only their own posts.

Step 10 — Profile Page
Profile Picture: Upload image → saved as Base64 in `fit_track_profile_img_{username}`.
Dark Mode: Toggle adds `dark-mode` class to body → CSS variables swap.
Export JSON: Downloads your activities + goals as a `.json` file.

Step 11 — Contact / Support
Fill in Name, Email, Message.
`UI.validateForm()` validates email format + required fields.
Shows success toast (no real backend).

Step 12 — Logout
Click "Logout" in sidebar → `Auth.logout()` removes `fit_track_current_user`.
Shows auth screen, destroys all charts.
Data persists — logging back in restores everything.

---

6. JavaScript Features Implemented

#	Feature	Implementation
1	**Form Validation**	`UI.validateForm()` — validates Activity, Goal, Contact forms (required fields, email, positive numbers)
2	**Smooth Scrolling**	Navigation click → content fades out → page switches → scrolls to top smoothly
3	**Interactive Image Slider**	Dashboard slider — auto-play 5s, manual prev/next, dot navigation
4	**Event Handling**	Tooltips on stat cards (mouseenter/mouseleave), hover effects, click events
	Dynamic Content	

5	**Updates**	Activity filtering, live chart updates, real-time stat recalculation
6	**Custom Animations**	CSS fadeIn keyframes, slider cubic-bezier transitions, card hover lifts

7. Technologies Used

Technology	Purpose
HTML5	Page structure and semantic elements
CSS3	Design system with custom properties, dark mode, responsive layout
JavaScript (ES6 Modules)	Application logic — modules, arrow functions, template literals, destructuring
Bootstrap 5.3	Grid system, form controls, modals, utility classes
Bootstrap Icons	Icon library
Chart.js	Bar and line charts for activity visualization
Google Fonts	Inter + Poppins typefaces
localStorage	Client-side data persistence